

Implementación de concurrencia en módulos

Por:

Johan Esteban Galeano Carvajal

Programa:

Tecnología en Desarrollo de Software

Curso:

Sistemas Operativos

Docente:

Yesid Montenegro García

Institución Universitaria Pascual Bravo

Medellín, Colombia

2026

Simulación de concurrencia en el SIGET

Problema productor-consumidor aplicado a sensores de tráfico (Python, hilos y semáforos)

1. Cómo se implementó

La simulación recrea un escenario sencillo del Sistema Inteligente de Gestión de Tráfico (SIGET). Tres sensores de intersección funcionan como hilos productores: cada cierto tiempo generan una lectura de tráfico (cantidad de vehículos y velocidad promedio) y la dejan en un buffer compartido que solo puede almacenar 5 elementos. Al mismo tiempo, dos módulos de análisis actúan como hilos consumidores: toman esos datos del buffer y los procesan para decidir si la vía está fluida o congestionada. En total corren 5 hilos al mismo tiempo sobre una única cola FIFO compartida, que se convierte en el recurso crítico a proteger para que no haya accesos simultáneos sin control.

2. Mecanismos de concurrencia usados

El control se hizo con tres semáforos POSIX (`threading.Semaphore` de Python), tal como pedía la actividad:

- **mutex (semáforo binario, empieza en 1):** asegura exclusión mutua sobre el buffer. Solo un hilo puede leer o escribir a la vez, evitando condiciones de carrera.
- **espacios_vacios (semáforo contador, empieza en 5):** lleva la cuenta de los espacios libres. Si el buffer se llena, bloquea al sensor para que no lo desborde.
- **espacios_llenos (semáforo contador, empieza en 0):** indica cuántos datos hay listos para procesar. Si el buffer está vacío, bloquea al módulo de análisis para que no lea de más. Así se cambia la espera activa por un bloqueo eficiente.

También se usó un Lock auxiliar y un Event para cerrar todo de forma ordenada: cuando los tres sensores terminan, se libera el semáforo `espacios_llenos` una vez por cada consumidor. Así los consumidores despiertan y terminan sin quedarse esperando datos que ya no van a llegar, evitando un interbloqueo al final.

3. Lo que se observó y se concluyó

Durante la ejecución, con registros que incluían hora y nombre del hilo, se comprobó que la ocupación del buffer nunca pasó de su capacidad máxima ni bajó de cero. Tampoco se perdió ni se duplicó ningún dato. La alternancia entre productores y consumidores en la salida muestra que sí hubo concurrencia real, no una ejecución secuencial disfrazada. Al final, todos los hilos terminaron de forma controlada y el buffer quedó vacío, sin hilos colgados esperando.

Esta solución aporta a la fiabilidad del SIGET porque demuestra que, incluso con varias fuentes de datos a ritmos distintos (los sensores) y varios consumidores procesando en paralelo (los módulos de análisis), se puede mantener la integridad de la información de

tráfico sin pérdidas, corrupción ni bloqueos. Eso es clave en una infraestructura crítica que debe operar de forma continua y en tiempo real. Usar semáforos como mecanismo de sincronización encaja bien en este tipo de arquitectura productor-consumidor, y el mismo patrón se podría escalar a más sensores o módulos de análisis sin tocar la lógica de sincronización.